Project Deep-Bluffing

Strategy Defense: A Monte-Carlo-Rollout Expected-Value Agent
for Heads-Up Limit Texas Hold'em

Saksham Rajawat
Roll Number: 250958

ICG Summer Project 2026-27

Abstract

This report documents the design of a from-scratch Heads-Up Limit Texas Hold'em (HULHE) engine and an autonomous decision-making agent, `CustomPokerBot`. The agent estimates its hand equity at every decision point using Monte Carlo rollouts against a uniformly random opponent range, and converts that equity estimate into a discrete FOLD/CALL/RAISE action using a pot-odds-based Expected Value (EV) decision rule with a game-theoretically motivated bluffing component. We describe the state representation, the hand-strength quantization method, the reward/EV mathematics, and report empirical win-rates (in bb/100, the standard poker unit) against three baseline opponents.

Contents

1 Problem Setting

Heads-Up Limit Hold'em (HULHE) is a two-player, imperfect-information, zero-sum extensive-form game. Unlike the perfect-information control tasks (e.g. cart-pole balancing) that preceded this capstone, an agent here can never directly observe the full game state – specifically, the opponent's two hole cards are hidden. Any state representation and decision procedure must therefore reason over a *distribution* of possible opponent hands rather than a single deterministic state.

1.1 Game parameters enforced by the arena

- Fresh 100-chip stack every hand (episodic; no carry-over).
- Blinds: Small Blind = 1 chip, Big Blind = 2 chips.
- Fixed-limit betting: +2 chips per raise pre-flop/flop (the *small bet*), +4 chips per raise turn/river (the *big bet*).
- 4-bet cap per street (1 initial bet + 3 raises); once reached, only CALL/FOLD remain legal.
- Turn order: Small Blind acts first pre-flop; Big Blind acts first on every subsequent street.

2 State Representation & Quantization

At each call to `get_action`, the agent receives:

$$(\text{hole\_cards},\ \text{community\_cards},\ \text{pot\_size},\ \text{stack\_size},\ \text{amount\_to\_call},\ \text{legal\_actions}).$$

Rather than hand-encoding a fixed-width feature vector (e.g. a one-hot encoding of 52 cards, as would be required to feed a neural network), we quantize the raw card state into a single scalar sufficient statistic: the **equity** $\hat{p} \in [0, 1]$, defined as

$$\hat{p} \;=\; \Pr(\text{win}) \;+\; \tfrac{1}{2}\Pr(\text{tie}),$$

estimated over the unknown remainder of the deck. This is a dimensionality reduction from a combinatorial card state (up to $\binom{52}{2} \cdot \binom{50}{5} \approx 1.98 \times 10^{10}$ possible hole-card/board combinations) down to one real number, while still being a *sufficient* statistic for the EV-maximizing decision under a uniform opponent-range assumption (Section 5).

2.1 Why a single-number quantization suffices

For a fixed pot size P and call amount c, calling is profitable precisely when

$$\text{EV}_{\text{call}} \;=\; \hat{p}\,(P + c) \;-\; (1 - \hat{p})\,c \;\geq\; 0 \quad\Longleftrightarrow\quad \hat{p} \;\geq\; \frac{c}{P + c} \;=:\; \text{pot\_odds}.$$

Because this inequality depends on the cards *only through* $\hat{p}$, no richer state representation is required for a threshold-based FOLD/CALL/RAISE policy – the equity is a sufficient statistic for the decision, which is why we invest our modeling effort in estimating $\hat{p}$ as accurately and cheaply as possible rather than in a larger hand-crafted feature vector.

3 Equity Estimation via Monte Carlo Rollouts

Let K be the set of known cards (own hole cards $\cup$ community cards so far) and $R = \text{Deck} \setminus K$ the remaining unseen cards, $|R| = 52 - |K|$.

3.1 Sampling procedure

For each of N simulated rollouts:

1. Draw 2 cards from R uniformly without replacement as the opponent's hypothetical hole cards.
2. Draw the remaining $5 - |\text{community}|$ cards from what is left of R to complete the board.
3. Evaluate both 7-card hands with the from-scratch hand evaluator (Section 4) and record a win, tie, or loss for the agent.

The Monte Carlo estimator is

$$\hat{p}_N \;=\; \frac{1}{N}\sum_{i=1}^{N}\Big(\mathbb{1}[\text{win}_i] + \tfrac{1}{2}\mathbb{1}[\text{tie}_i]\Big),$$

which is unbiased for the true equity p under a uniform-random opponent-range assumption, with standard error $\sigma \approx \sqrt{p(1-p)/N}$ (e.g. $N = 200 \Rightarrow \sigma \lesssim 0.035$ at $p = 0.5$), which is small relative to the decision-threshold gaps used in Section 5.

3.2 Exact enumeration on the river

On the river, the community cards are fully known, so the *only* remaining uncertainty is the opponent's 2 hole cards – at most $\binom{45}{2} = 990$ combinations. Because this is cheap, the agent switches from sampling to *exact enumeration* on the river, eliminating all Monte Carlo variance exactly when the decision matters most (the pot is largest and there are no future cards to change the outcome).

3.3 Simulation budget per street

To keep the agent fast enough for a 10,000-hand tournament while controlling variance, the number of rollouts is scaled with the number of unknown board cards (Table 1).

Street	Unknown board cards	Rollouts N	Method
Pre-flop	5	120	Monte Carlo sampling
Flop	2	200	Monte Carlo sampling
Turn	1	300	Monte Carlo sampling
River	0	990 (exact)	Full enumeration

Table 1: Rollout budget per street.

4 The Hand Evaluator

No external poker library (e.g. `treys`, `RLCard`) is used. The evaluator finds the best 5-card hand out of the 7 available cards using only `itertools` and `collections.Counter`.

4.1 Reference implementation

A brute-force reference evaluator enumerates all $\binom{7}{5} = 21$ five-card subsets via `itertools.combinations`, scores each with a `Counter`-based rank/suit tally, and keeps the maximum. This is correct by construction but runs in $O(21)$ per call.

4.2 Optimized $O(n)$ evaluator

Because the agent's Monte Carlo rollouts call the evaluator tens of thousands of times during a single hand, a faster direct-counting evaluator was written that inspects the 7 cards once (no subset enumeration): it tallies suit and rank counts with `collections.Counter`, checks for a flush suit and a straight (handling the wheel straight A-2-3-4-5 by re-inserting the Ace as rank 1) directly on the relevant rank sets, and falls through the standard hand hierarchy (straight flush $\rightarrow$ quads $\rightarrow$ full house $\rightarrow$ flush $\rightarrow$ straight $\rightarrow$ trips $\rightarrow$ two pair $\rightarrow$ pair $\rightarrow$ high card). This reduces the per-call cost roughly $21\times$ relative to the brute-force version.

4.3 Correctness verification

The optimized evaluator was cross-validated against the brute-force reference implementation on 20,000 independently sampled random 7-card hands (`test_evaluator.py`), plus hand-written edge cases: the wheel straight, the royal flush, quads-beats-full-house, and board-plays split pots. All 20,000 random hands and all edge cases matched exactly.

5 EV-Based Decision Rule

Given equity $\hat{p}$, pot size P, and call amount c, define pot_odds $= c/(P + c)$ as in Section 3. The agent's policy is:

1. **Value raise:** if `RAISE` is legal and $\hat{p} \geq \theta_{\text{raise}} = 0.68$, RAISE. This is played whenever the hand is a clear statistical favorite, in order to build the pot while ahead (maximizing $\mathbb{E}$[winnings], not just win probability).
2. **Balanced semi-bluff:** if `RAISE` is legal and $\hat{p} \in [0.32, 0.45]$, RAISE with probability $\beta = 0.30$. This mixed strategy exists so the agent cannot be exploited by an opponent who simply folds to all aggression or, conversely, by one who always calls – a pure threshold strategy with no randomization in this band is trivially exploitable in the game-theoretic sense, whereas a mixed strategy forces the opponent to respect some raises even when the agent's true equity is moderate.
3. **Call:** if facing a bet ($c > 0$) and $\hat{p} \geq$ pot_odds $+ \varepsilon$ (with a small safety buffer $\varepsilon = 0.02$ against near break-even spots and Monte Carlo noise), CALL.
4. **Fold:** if facing a bet and $\hat{p} <$ pot_odds $+ \varepsilon$, FOLD (negative expected value to continue).
5. **Check:** if no bet is facing the agent ($c = 0$) and neither of the two raising conditions triggered, CALL (which the arena defines as a CHECK when `amount_to_call` is 0).

5.1 Why pot odds is the correct break-even criterion

If the agent calls and wins the hand with probability $\hat{p}$, its net gain is P (the pot *before* its own call is added) with probability $\hat{p}$, and its net loss is c (the call amount) with probability $1 - \hat{p}$. Hence

$$\text{EV}_{\text{call}}(\hat{p}) \;=\; \hat{p}\,P \;-\; (1 - \hat{p})\,c.$$

Setting $\mathrm{EV}_{\mathrm{call}} \geq 0$ and solving for $\hat{p}$ gives

$$\hat{p} \;\geq\; \frac{c}{P+c} \;=\; \text{pot\_odds},$$

which is exactly the break-even criterion used in Steps 3-4 of the decision rule above.

6 Engine Design Notes

The engine (`engine.py`) implements a heads-up betting-round state machine with two players indexed by button position, tracking per-street contributions and a bet counter capped at 4. The round ends when both players have acted since the last raise *and* their contributions are equal – this correctly reproduces the "big blind option" pre-flop (the BB may still raise even after the SB calls) and ordinary check-check/bet-call sequences post-flop. Showdown resolves ties by splitting the pot evenly (odd chip to the button, a standard convention).

6.1 Known simplification

The engine does not implement side pots for uneven all-in stacks (a player who cannot cover a full call/raise is capped to what remains in their stack). Given the 100-chip starting stack and the 4-bet cap per street, this scenario is rare in practice and was judged out of scope for a heads-up (single main pot only) engine; it is flagged here for transparency rather than left silent.

7 Experimental Results

`test_arena.py` plays the agent against three baseline opponents for several thousand isolated hands each, alternating the button every hand. Table 2 reports the standard poker performance metric bb/100 (big blinds won per 100 hands).

Opponent	Hands	Net chips	bb/100
RandomBot	2,000	+4,225	+105.6
CallingStationBot	2,000	+3,056	+76.4
TightAggressiveBot	2,000	+9,013	+225.3

Table 2: CustomPokerBot performance vs. baseline opponents. Zero invalid actions were returned across all runs.

The agent is profitable against all three archetypes. It beats `TightAggressiveBot` most heavily because that baseline raises unconditionally and folds to any bet, both highly exploitable patterns that a pot-odds-aware caller/raiser punishes directly. Its edge over `CallingStationBot` is smaller, as expected: a bot that never folds cannot be bluffed, so all edge there must come from value betting with genuinely stronger hands.

7.1 Runtime

With the optimized $O(n)$ evaluator, the agent plays roughly 20–40 ms/hand depending on opponent aggression (more raises mean more `get_action` calls per hand), which comfortably fits within a 10,000-hand tournament.

8 Limitations & Future Work

- The opponent model is a uniform random range; a stronger agent could maintain a Bayesian belief over opponent range narrowed by observed betting actions (e.g. treat a river raise as evidence of a stronger range and re-weight the Monte Carlo opponent sampling accordingly).
- The bluffing frequency β and thresholds $\theta_{\text{raise}}, \theta_{\text{bluff}}$ were set from poker-theory intuition and lightly tuned against the baselines in Table 2; a natural extension is to learn them via self-play (e.g. CFR self-play regret minimization or policy-gradient RL over the equity-conditioned policy), rather than fixing them by hand.
- Side pots for uneven all-in stacks are not modeled (see Section 6.1).

9 Conclusion

`CustomPokerBot` combines a from-scratch, formally verified hand evaluator with Monte Carlo equity rollouts and a pot-odds-derived EV decision rule, including a small randomized bluffing component for game-theoretic robustness. The resulting agent is profitable against three qualitatively different baseline strategies while returning zero invalid actions across thousands of test hands, and runs fast enough for the 10,000-hand tournament format.